

Project 3 Report

Project Overview/Goals:

In this project, we made a world where there's little critters that move around a grid and they fight each other. Each critter has its own set of instructions that uses a critter language. We had three goals for this project.

- We wanted to build an interpreter that reads and runs the set of instructions in the critter language.
 - loadSpecies reads the critter file and saves the critters name and the instructions
 - executeCritter runs the critters instructions
- We built a test harness so we could test our interpreter in situations that we could make instead of the random simulator
- We designed a critter

Scope/Quality/Limitations

We understood that all the instructions in the spec are working which included moving, turning, eating ... and all commands. We also took that the jumps go to the exact line we instruct it to and that the registers work and can be written from r1-r10 and it works

In our known limitations, right now we don't have register jumpers which means we can jump to a line number that is stored in a register like go to r2 or something like that, but our interpreter doesn't work for that. Our critter that we designed doesn't really use that so we should be fine.

Also our line numbers start at 0 so the critter skips its very first instructions but we found out that it eventually works

Also if there is a critter file that is faulty, we don't reject it and we also have a loop program which means that if we have one line that says go 1, it loops forever and the simulation just freezes

Assumptions

The first line of the file is always the critter's name and then after that it's all instructions

Registers are numbered from 1-10

When there isn't a critter in the spot ifangle checks, then that check is always false.

Algorithm and Implementation

Lets look at how loadSpecies works:

- It basically reads the whole file into a long string
- It then cuts the string into a blank line and then disregards everything after it in a sense
- It splits what's in the lines again and then the first line is the name like we mentioned, and then the rest of the lines becomes instructions
- We then return a new CritterSpecies with the name and list

Next let's see how executeCritter works:

- It gets the line to run from the critter and then if its at the end of the program, we stop.

- We split our instructions into words and the first word we have is our command which instructs our critter
- The actions commands do the action and then we save the line number that comes after that line
- Other commands like the if checks changes the line we go to, and its like a loop which keeps going until an action happens

There's 2 helper methods

jumpCalculator figures out our jump from line to line, and +n goes forward and -n goes backward. Just a number is an exact line and then we have to subtract 1 because our code always starts from 0.

parseArgs checks if an instruction for our critter has the right number of parts or in this case "arguments" and we then turn it into numbers so it can understand what 'r1 or r2' means.

The if commands work the same as well. We check the condition and then either jump or go to the next line.

Our critter is called Kodukula and basically we fight forward and then we run from behind.

So basically how it works is our critter counts the turns since it last ate using r1 because eating is very important, because the critter moves slow if it doesn't have enough food. After that if theres an enemy in front of it, it eats the enemy if the enemy is facing away from us if the critter is hungry, or if its been more than 30 turns since we last ate, which we

count using r1. If it doesn't do this, then we just infect it and make that critter join our team

After that it turns toward enemies in front of it or ones that are on its right that aren't aiming because then next turn it can attack them.

Then when the enemies get mad and try to attack it, it runs away by hopping out of the way, and if it's stuck, then it turns, but turning to face the enemy critter is very slow and the enemy would attack our critter

To save turns, we made it so that our critter ignores the enemies that are behind it, because turning is just a waste of moves

Finally, we hop forward when nothing is around so instead of spinning, it covers more of the grid.

We built other test critters to make our critter stronger and a critter that specifically dodged attacks and a critter that always ate before it was hungry beat our critter on one on ones but we rebuilt our critter so in crowded areas where other critters try to get food too, our critter fares better

Testing

The simulator we have is bad for testing because it's random and you can't repeat the same situation for our critter to find the bug. Also you can't create a specific situation. What I mean by that is if the enemy is behind me, or if its facing me or if its starving. I could only hope that situation occurred again. Also you see all the critters moving at once, so

I can't see the decisions one critter makes (well I can, but tracking it really sucks because they move super fast)

So our test harness allowed us to make a FakeCritic which we could control. So we had code to see how the fake critter would move and then we controlled what was around it and the way the critters were facing and the hunger level and the registers. This way we had total control of the situation. We also checked the action that it did and what it infected with and the lines which will run, so it allowed us to see the decision making process

Our test cases check for the actions the critter makes, the go jumps forward and backward, the random, the hunger level, the starving level, the space checks through the if conditions and then the register commands. We also had jump calculator to check if it was going to the correct line or not.

We believe we ran over 250 simulations between our critter and test critters. And we had one-on-one games which let us know about the weaknesses that I mentioned before but also how they really don't matter in crowded games.

Interesting Results

Some thing we found out was that our critter was actually very good because it ended up with the largest share of critters and us changing each line instruction this entire past week allowed our critter to become better and better.

The one thing I will say is how our critter sometimes lost against the "eat before hungry" test critter and the "always attack" critter during 1 v 1s, but in a crowded place, it managed to win, so that made Vish and I

think about how sometimes winning isn't really just brute force or energy.

We realized speed is super important because a hungry critter almost always lost.

And honestly I was thinking about this, our critter really doesn't have a teamwork rule thing. It like slowly infects other critters and then they slowly make the teams for our critter.

Problems Encountered

The problems we had was the counting lines as well as the register names. What I mean by this is that our code counts from 0 while the simulator counts from 1. So there was a lot of "off by one" bugs but it eventually lined up and started to work so we think that's partially resolved.

The register names like r1 which should be an instruction that stands by itself but our code used inc r1, so we had to change the interpreter so that it accepts the r in the register instructions.

And finally one thing was that this week was really busy and testing our critter against all the other critters took a lot of games and simulations and we ran over 200 simulations on maps and the random part of the simulation almost made it a luck based game.

Pair Programming:

Monday: Anirudh drives 1 hour. Vish drives 3 hours.

Tuesday: Vish drives 2.5 hours.

Wednesday: Vish alone 1 hour.

Thursday: Anirudh alone 2 hours.

Friday: Anirudh alone 4 hours. Vish alone 30 minutes.

Pair programming was honestly super fun for us, because first of all we were good friends so coding this solution and thinking about our critter was super cool. We also had some silly times where we would come up with absurd strategies and just go on tangents about that. It was different from working alone because we had each other to always discuss all our thoughts and approach with fully and it let us understand how we think about these coding problems. We were able to catch each others mistakes and correct it and that was super efficient for our code. One thing that was hard was that at times we would go on tangents and waste some time, but we ultimately made it back on track and this experience was super cool.

Social Impact

CritterFest was so fun and it became a competition between CSB and Turing (shhh we heard about through the grapevine) and honestly it had its pros and cons. The pros include competition fueling great progress, and honestly it was such a fun experience being able to create a critter that will “win the entire competition.” But competition also makes people secretive I’d say because our strategies were technically competing against each other. Good teamwork allowed us to become closer and this competition based programming is prevalent in the real world as well I’d say because it’s just such a good way to fuel progress and that’s what our jobs do as well. We get paid to compete against others in the same industry.